Title: Smart Dictionary with Intelligent Word Search Using Binary Search Tree (BST) and AVL Tree

1. Problem Statement

The aim of this project is to develop a Smart Dictionary using Binary Search Tree (BST) and AVL Tree. The application allows users to add, search, delete, and display words along with their meanings, synonyms, and antonyms.

It also provides features such as displaying words in alphabetical order, performing tree traversals, finding the first and last words, comparing BST and AVL search performance, displaying tree height and total word count, and showing a Word of the Day.

2. Objectives

- To understand and implement the concepts of Binary Search Tree (BST) and AVL Tree.
- To develop a dictionary application using tree data structures.
- To perform insertion, searching, and deletion of words.
- To display dictionary words in alphabetical order.
- To implement Inorder, Preorder, and Postorder traversals.
- To find the first and last words in the dictionary.
- To compare the searching performance of BST and AVL Tree.
- To calculate tree height and total number of words.
- To provide a simple and user-friendly menu-driven application.

3. Algorithm

Step 1: Start

Initialize the BST and AVL Tree as empty.

Step 2: Display Menu

Display the available operations and accept the user's choice.

Step 3: Add Word

Accept the word, meaning, synonym, and antonym.

Check whether the word already exists.

If it does not exist, insert it into both BST and AVL Tree.

Step 4: Search Word

Accept a word from the user.

Search for it in the dictionary.

If found, display its meaning, synonym, and antonym.

Otherwise, display a word-not-found message.

Step 5: Delete Word

Accept the word to be deleted.

Search for the word.

If found, delete it from both trees.

Step 6: Display Words

Use Inorder Traversal to display words in alphabetical order.

Step 7: Perform Tree Traversals

Display Inorder, Preorder, and Postorder traversals for both trees.

Step 8: Find First and Last Words

Find the minimum and maximum words in the BST.

Step 9: Compare BST and AVL Tree

Display the number of search comparisons and the height of both trees.

Step 10: Display Statistics

Display the total number of words and the height of both trees.

Step 11: Word of the Day

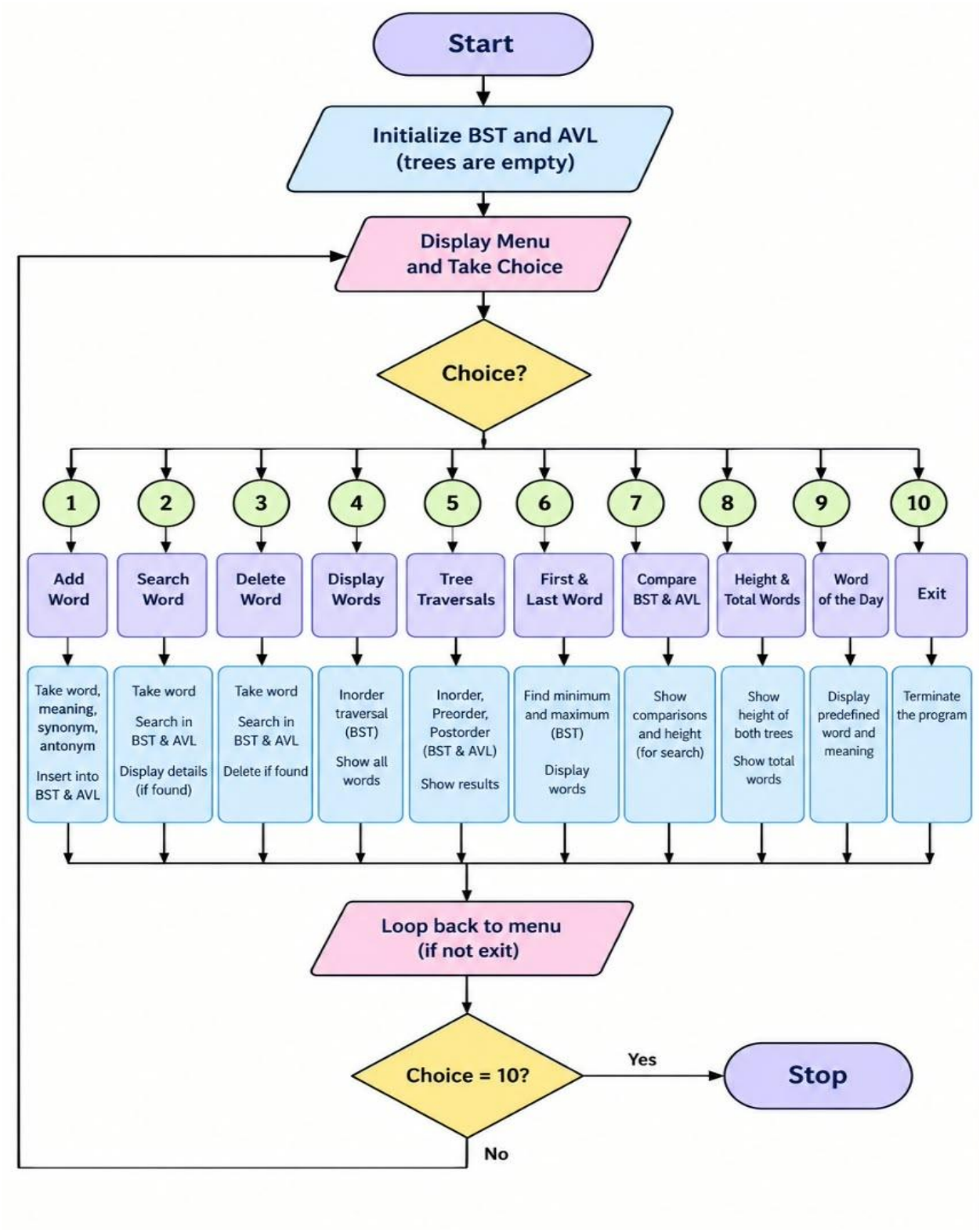
Display a predefined word and its meaning.

Step 12: Repeat or Exit

Return to the menu if the user wants to continue.

Stop the program when the user selects Exit.

4. Flowchart


Start
Initialize BST and AVL (trees are empty)
Display Menu and Take Choice
Choice?
1
2
3
4
5
6
7
8
9
10
Add Word
Search Word
Delete Word
Display Words
Tree Traversals
First & Last Word
Compare BST & AVL
Height & Total Words
Word of the Day
Exit
Take word, meaning, synonym, antonym
Insert into BST & AVL
Take word
Search in BST & AVL
Display details (if found)
Take word
Search in BST & AVL
Delete if found
Inorder traversal (BST)
Show all words
Inorder, Preorder, Postorder (BST & AVL)
Show results
Find minimum and maximum (BST)
Display words
Show comparisons and height (for search)
Show height of both trees
Show total words
Display predefined word and meaning
Terminate the program
Loop back to menu (if not exit)
Choice = 10?
Yes
Stop
No

5.Conclusion

The Smart Dictionary project demonstrates the practical application of Binary Search Tree (BST) and AVL Tree in a dictionary management system. It allows users to add, search, delete, and display words efficiently.

This project helps in understanding important data structure concepts such as tree insertion, deletion, searching, traversal, height calculation, balancing, and rotations. It also demonstrates the difference between BST and AVL Tree through search comparisons and tree height.